```
# TEST 2 (8D-internal): from the relation table alone, derive the sector split, the
pairing (no phi used),
# uniqueness of partners, transport under table symmetries, and the exact level
values. Compare to earlier results only at the end.
import numpy as np, io, contextlib, itertools, sympy as sp
src=open('45_table_only_cycle_frame.py').read()
ns={}
with contextlib.redirect_stdout(io.StringIO()): exec(src,ns)        # table G,
internal coefficients C, cycles res[seed], conjugator()
G,C,GB,N,res,channels,sector_sq,conjugator=[ns[k] for k in "G C GB N res channels
sector_sq conjugator".split()]
pairs_of_ch={1:[1,29],7:[7,23],11:[11,19],13:[13,17]}
def analyse(perm):
    ms,Pm=channels(perm)
    # (a) all six ways to split the four channels 2+2 : which give exactly two
levels?
    splits={}
    for A in itertools.combinations([1,7,11,13],2):
        if 1 not in A: continue
        Bc=tuple(k for k in [1,7,11,13] if k not in A)
        lv=np.round(sector_sq(Pm,sum((pairs_of_ch[k] for k in A),[])),9)
        splits[(A,Bc)]=sorted(set(lv))
    return Pm,splits
def pairing(Pm,A):
    P=sum(Pm[m] for k in A for m in pairs_of_ch[k]); X=np.real(C@P.T)            #
sector components, internal coefficients
    nrm=np.sqrt(np.einsum('ri,ij,rj->r',X,GB,X)); U=X/nrm[:,None]
    cos=U@GB@U.T; lv=np.round(nrm**2,9); L=sorted(set(lv)); inner=np.where(lv==L[0])
[0]; outer=np.where(lv==L[1])[0]
    same=np.abs(cos-1)<1e-9                                                    #
same direction in the sector
    cnt=[int(same[i,outer].sum()) for i in inner]; cnt2=[int(same[o,inner].sum()) for
o in outer]
    part={int(i):int(outer[np.where(same[i,outer])[0][0]]) for i in inner} if
set(cnt)=={1} else None
    return part,inner,outer,cnt,cnt2,nrm
p1=res[0][1]; Pm,splits=analyse(p1)
print("(a) six 2+2 groupings of channels (labels 1,7,11,13 with conjugates) ->
distinct squared sector values:")
for k,v in splits.items(): print("   ",k,":",len(v),"value(s)", v if len(v)<=3 else
"")
two=[k for k,v in splits.items() if len(v)==2]; print("   groupings giving exactly
two values:",two)
A=two[0][0]
part,inner,outer,cnt,cnt2,nrm=pairing(Pm,A)
print(f"\n(b) pairing rule: different value, same direction in sector {A} (cosine
```

```
within 1e-9). No ratio used.")
print("    partners per inner element:",sorted(set(cnt)),"| partners per outer
element:",sorted(set(cnt2)),"| pairs:",len(part) if part else None)
print("    each outer element used exactly once:",part is not None and
len(set(part.values()))==len(outer))
print("    relation value between partners:",sorted(set(int(G[i,o]) for i,o in
part.items())))
print("    cycle carries pairs to pairs (partner(c r)=c
partner(r)):",all(part[int(p1[i])]==p1[o] for i,o in part.items() if int(p1[i]) in
part))
# (c) transport under table symmetries relating cycle 0 to cycles 1..11
print("\n(c) transport: the table symmetry s relating cycle 0 to cycle k")
for seed in range(1,12):
    pk=res[seed][1]; s=conjugator(p1,pk); S=np.array([s[r] for r in range(N)])
    Pk,_=analyse(pk); partk,*_=pairing(Pk,A)
    moved={int(S[i]):int(S[o]) for i,o in part.items()}
    print(f"   seed {seed:2d}: s carries pairing 0 onto pairing {seed}:
{moved==partk} | pairs shared by pairing 0 and pairing {seed}:
{len(set(part.items())&set(partk.items()))}/120")
# (d) ratios read off afterwards, numerically
ratio=np.array([nrm[o]/nrm[i] for i,o in part.items()]); print("\n(d) partner length
ratio in the sector: min %.15f max %.15f"%(ratio.min(),ratio.max()))
r=ratio.mean(); print("    r^2 - r - 1 = %.2e"%(r*r-r-1))
rho=lambda k: np.sqrt(np.real(np.einsum('ri,ij,rj->r',*( [np.real(C@sum(Pm[m] for m
in pairs_of_ch[k]).T)]*1 ),GB,np.real(C@sum(Pm[m] for m in pairs_of_ch[k]).T)) if
False else None))
X={k:np.real(C@sum(Pm[m] for m in pairs_of_ch[k]).T) for k in pairs_of_ch}
rh={k:np.sqrt(np.einsum('ri,ij,rj->r',X[k],GB,X[k])) for k in X}
Bc=two[0][1]
for k in [1,7,11,13]:
    rr=[rh[k][o]/rh[k][i] for i,o in part.items() if rh[k][i]>1e-12]; print(f"
channel {k}: partner/element magnitude ratio, distinct values
{sorted(set(np.round(rr,12)))}")
prod=max(abs(rh[a][i]*rh[b][i]-rh[a][o]*rh[b][o]) for i,o in part.items() for a in A
for b in Bc)
print("    cross products (one channel from each group) preserved: max change
%.2e"%prod)
# (e) exact values: the sector is the kernel of one rational-sqrt5 factor of the
cycle's characteristic polynomial
M=sp.Matrix(np.array([C[p1[b]] for b in ns['B']]).T.tolist()).applyfunc(lambda v:
sp.nsimplify(round(v*840))/840)
x=sp.symbols('x'); chi=sp.factor(M.charpoly(x).as_expr(),extension=sp.sqrt(5));
print("\n(e) characteristic polynomial of the cycle, factored over Q(sqrt5):",chi)
GBs=sp.Matrix(np.round(GB).astype(int).tolist())
for f in sp.Mul.make_args(chi):
    cf=sp.Poly(f,x).all_coeffs(); FM=sp.zeros(8,8)
    for c_ in cf: FM=(FM*M+c_*sp.eye(8)).applyfunc(sp.radsimp)
    K=sp.Matrix.hstack(*FM.nullspace(simplify=True))
    Q=(GBs*K*(K.T*GBs*K).inv()*K.T*GBs).applyfunc(sp.radsimp)
    vals=set()
```

```
    for rr in range(N):
        c=sp.Matrix([sp.nsimplify(round(v*840))/840 for v in C[rr]]);
vals.add(sp.radsimp(sp.expand((c.T*Q*c)[0])))
    print("    factor",f,": exact squared sector values",sorted(vals,key=float))
```